Session 08 — Project Guide: Collection State + CRUD

This guide explains the actual official Session End for Session 08 in depth. It does not include homework solutions — homework asks you to extend this exact code yourself.

If you forgot most of class, read this guide slowly, section by section, and you should be able to rebuild the mental model from nothing.

---

1. Where today's session starts

Session 08 starts from Session 07's **full homework solution** — the version of TaskFlow that already has named person-filter buttons that work, an empty-result message, and a derived `peopleWithCounts` list, on top of everything from Session 06 (state, events) and Session 05 (components, props). If any of that sounds unfamiliar, re-read the Session 07 project guide first — this guide assumes it.

2. The problem with Session 07's `tasks`

In Session 07, `tasks` looked like this, sitting OUTSIDE any component, at the top of the file:

```
const tasks: Task[] = [
  { id: 1, userId: 1, title: "Finish JavaScript exercise", completed: false },
  { id: 2, userId: 2, title: "Review pull request", completed: true },
  { id: 3, userId: 3, title: "Write session notes", completed: false },
  { id: 4, userId: 1, title: "Update project README", completed: true },
  { id: 5, userId: 2, title: "Fix search bug", completed: false },
  { id: 6, userId: 3, title: "Plan sprint review", completed: true },
];
```

This is a plain `const` array. You could read it — `.map()` it into a list, `.filter()` it into `visibleTasks`, `.reduce()` it into stats — because reading an array never changes it. But you could never Add, Toggle, Delete, or Edit a task, because:

- **`const` does NOT make the array immutable — it only stops the VARIABLE `tasks` from being reassigned.** You already proved this in the JS/TS phase: `const list = [1, 2, 3]; list.push(4);` works fine, because `.push()` mutates the array `list` still points to — it never reassigns `list` itself. So yes, `tasks.push(newTask)` would technically run without a TypeScript or runtime error.

- **That was never the real blocker.** A plain JavaScript variable — whether you mutate it or leave it alone — never tells React "something changed, please render the screen again." React only re-renders in response to a **state** update, made through a setter function. A normal variable, `const` or not, mutated or not, is completely invisible to React.

This is exactly the same limitation `useState` already solved for single values back in Session 06 (`currentFilter`, `searchText`, `selectedUserId`) — except now the same problem needs solving for an entire **collection**, not just one value.

3. `useState<Task[]>` — state that holds a collection

The fix, in the actual Session End source (`src/App.tsx`):

```
const initialTasks: Task[] = [
  { id: 1, userId: 1, title: "Finish JavaScript exercise", completed: false },
  { id: 2, userId: 2, title: "Review pull request", completed: true },
  { id: 3, userId: 3, title: "Write session notes", completed: false },
  { id: 4, userId: 1, title: "Update project README", completed: true },
  { id: 5, userId: 2, title: "Fix search bug", completed: false },
  { id: 6, userId: 3, title: "Plan sprint review", completed: true },
];
```

```
function App() {
  const [tasks, setTasks] = useState<Task[]>(initialTasks);
  const [currentFilter, setCurrentFilter] = useState<FilterStatus>("all");
  const [searchText, setSearchText] = useState("");
  const [selectedUserId, setSelectedUserId] = useState(0);
```

Line by line:

- `initialTasks` — the SAME six starting tasks Session 07 had, just renamed. It exists only to give `useState` a starting value; nothing reads `initialTasks` again after this line.
- `const [tasks, setTasks] = useState<Task[]>(initialTasks);` — read this exactly the way you already read `useState<FilterStatus>("all")` in Session 07: `useState<T>` says WHAT TYPE of value this state holds. Here, `T` is `Task[]` — an array of `Task` objects, not a single value. `tasks` is the current array; `setTasks` is the only correct way to replace it; `initialTasks` is the starting value, used once.

Connecting it to what you already know: a scalar `useState("all")` and a collection `useState<Task[]>(initialTasks)` are the same tool. The only difference is what's inside the box — Session 06 put one string in the box; Session 08 puts a whole array in the box. Every rule

you already learned about state (never write to it directly, always use the setter, a setter call triggers a re-render) applies identically.

4. The immutable update mental model

This is the single most important idea in this session. Say it out loud:

```
ACTION (Add / Toggle / Delete / Edit)
  -> CREATE A BRAND-NEW ARRAY (never edit the old one)
  -> setTasks(newArray)
  -> REACT RENDERS App AGAIN
  -> visibleTasks / totalCount / peopleWithCounts recalculate automatically,
     because Session 07 already made them DERIVED from tasks
```

Compare this directly to the already-known Session 04C loop:

```
ACTION -> CHANGE DATA/STATE -> RENDER AGAIN -> UI CHANGES
```

The shape is identical. The two real differences are:

1. **Session 04C mutated data in place** (`tasks.push(newTask)`) and called a manual render function itself. **React state is never mutated** — every update builds a brand-new array or object, and the old one is left completely untouched.
2. **Session 04C called a render function by hand.** With React state, nobody calls a render function — calling `setTasks(...)` is the signal; React decides when and how to render again.

Why immutability matters, not just "because the rules say so": React decides whether to re-render partly by comparing the OLD state value to the NEW one. If you mutated the same array object in place and then called `setTasks(tasks)`, React could see the exact same array reference it already had and have no reliable way to know something inside it changed. Building a brand-new array every time removes that ambiguity completely.

Mental model diagram

```
copied over
   └──────────────────────┘              └──────────────────────┘

   never touched again                  this is what React now renders from
```

5. Array spread — Add Task

The actual Session End `handleAddTask`, in `src/App.tsx`:

```
function handleAddTask(title: string, userId: number) {
  const newTask: Task = {
    id: Date.now(),
    userId,
    title,
    completed: false,
  };

  setTasks([...tasks, newTask]);
}
```

- `id: Date.now()` — gives the new task a number that has never been used before (the current timestamp in milliseconds), playing the same role `task.id` already plays for the six starting tasks.
- `[...tasks, newTask]` — this is **array spread**, brand-new syntax this session. `...tasks` means "copy every item currently inside `tasks`, one by one, into this new array." Then `, newTask` adds one more item at the end. The result is a completely new array — `tasks` itself is never touched.
- `setTasks(...)` — hands that new array to React and asks for a re-render.

Common mistake:

```
// WRONG
tasks.push(newTask);
setTasks(tasks);
```

`.push()` mutates the existing array in place. Even though `setTasks` is still called, the ARRAY REFERENCE passed to `setTasks` is the exact same one React already had, which defeats the whole point of immutable updates. Always build a new array with spread (or `.map()` / `.filter()`), never mutate the old one.

6. Where the new task comes from — `AddTaskForm`

A new component, `src/components/AddTaskForm.tsx`:

```
interface AddTaskFormProps {
  users: User[];
  selectedUserId: number;
  onAddTask: (title: string, userId: number) => void;
}

function AddTaskForm(props: AddTaskFormProps) {
  const defaultUserId = props.selectedUserId || props.users[0].id;

  const [draftTitle, setDraftTitle] = useState("");
  const [draftUserId, setDraftUserId] = useState(defaultUserId);

  function handleTitleChange(event: ChangeEvent<HTMLInputElement>) {
    setDraftTitle(event.target.value);
  }

  function handleUserChange(event: ChangeEvent<HTMLSelectElement>) {
    setDraftUserId(Number(event.target.value));
  }

  function handleSubmit(event: FormEvent) {
    event.preventDefault();

    const trimmedTitle = draftTitle.trim();

    if (trimmedTitle === "") {
      return;
    }

    props.onAddTask(trimmedTitle, draftUserId);

    setDraftTitle("");
    setDraftUserId(props.selectedUserId || props.users[0].id);
  }

  return (
    <form className="add-task-form" onSubmit={handleSubmit}>
```

```
      <input
        type="text"
        className="add-task-input"
        placeholder="Add a new task..."
        value={draftTitle}
        onChange={handleTitleChange}
      />
      <select
        className="add-task-select"
        value={draftUserId}
        onChange={handleUserChange}
      >
        {props.users.map((user) => (
          <option key={user.id} value={user.id}>
            {user.name}
          </option>
        ))}
      </select>
      <button type="submit" className="add-task-button">
        Add Task
      </button>
    </form>
  );
}
```

Line by line, focusing on what's new:

- `<form className="add-task-form" onSubmit={handleSubmit}>` — `onSubmit` is a new event, but the same shape as every event you already know: it fires when the form is submitted (clicking the "Add Task" button, or pressing Enter inside the text input — that's a normal browser form behavior, not something this code writes). `handleSubmit(event: FormEvent)` — `FormEvent` is React's typed event for form submissions, the same idea as `ChangeEvent<HTMLInputElement>` for input changes, just for a different kind of event.
- `event.preventDefault();` — by default, submitting an HTML form makes the browser reload the page. That would wipe out all of React's state instantly. `preventDefault()` stops that default browser behavior so the rest of `handleSubmit`'s JavaScript can run instead.
- `function handleUserChange(event: ChangeEvent<HTMLSelectElement>)` — the exact same `ChangeEvent<T>` pattern already used for the text input (`ChangeEvent<HTMLInputElement>`), applied to a `<select>` element instead. The type argument just names which HTML element fired the event.

- `onAddTask: (title: string, userId: number) => void` — this is a **callback prop**: a function type, passed down as a prop, that the child calls when it wants to ask the parent to do something. `AddTaskForm` has no access to the real `tasks` state — it isn't allowed to and doesn't need to. It just calls `props.onAddTask(trimmedTitle, draftUserId)` and trusts whoever gave it that function (in this case, `App`) to do the actual work.
- `draftTitle` / `draftUserId` — this form's OWN local state, describing what the user is currently typing/selecting. This is completely separate from the real `tasks` array — it only becomes a real task the moment `onAddTask` is called.
- `defaultUserId = props.selectedUserId || props.users[0].id` — the "smart default owner." If a specific person is already selected in the page's person filter (`selectedUserId` is not `0`), default the new task's owner to them. Otherwise (`selectedUserId` is `0`, meaning "All people"), fall back to the first real user. `0` is falsy in JavaScript, so `||` reads naturally here — this is the same falsy-fallback idea already used elsewhere in the course, applied to a new situation.
- **Be precise about WHEN this default is computed — it is not continuously synced.** `useState(defaultUserId)` only reads `defaultUserId` ONCE, the moment `AddTaskForm` first mounts — that is how every `useState` initializer already works. If the person filter changes later while the same `AddTaskForm` is still on screen without the form being resubmitted, `draftUserId` does **not** silently follow it — there is no `useEffect` here watching for that. The default is recomputed at exactly one other moment: right after a successful submit, where `handleSubmit` explicitly calls `setDraftUserId(props.selectedUserId || props.users[0].id)` again by hand. Two explicit moments, not a continuous sync — this is a deliberate, `useEffect`-free design, not an accident.

Connecting to what you already know: this form's shape — controlled inputs, a `handleSubmit` guard before doing the real work — is the exact same shape as every controlled input you've built since Session 06. The only genuinely new piece here is that this component talks back to its parent through a function it was given, instead of managing everything itself.

Wiring it up, in `App`:

```
<AddTaskForm
  users={users}
  selectedUserId={selectedUserId}
  onAddTask={handleAddTask}
/>
```

`onAddTask={handleAddTask}` is the connection: `App` hands its OWN `handleAddTask` function down as a prop. When `AddTaskForm` calls `props.onAddTask(...)`, it is really calling `handleAddTask`, back here in `App`, where the real `tasks` state lives.

7. Object spread — Toggle Completed

```
function handleToggle(id: number) {
  const updatedTasks = tasks.map((task) => {
    if (task.id === id) {
      return { ...task, completed: !task.completed };
    }

    return task;
  });

  setTasks(updatedTasks);
}
```

- `.map()` — already known since Session 07, but used differently here: instead of mapping data into JSX, it's mapping the OLD array into a NEW array of the same length, where every item is either passed through unchanged or replaced.
- `{ ...task, completed: !task.completed }` — this is **object spread**, the sibling of array spread. `...task` copies every field already on `task` (`id`, `userId`, `title`, `completed`) into a brand-new object. Then `completed: !task.completed` overwrites just that ONE field on the copy. The original `task` object is never touched.
- `return task;` — for every task that ISN'T the one being toggled, pass it through completely unchanged. Forgetting this line is a common bug — without it, every non-matching task becomes `undefined`.

Mental model: object spread

```
task = { id: 3, userId: 1, title: "Write notes", completed: false }

{ ...task, completed: !task.completed }
        │                   │
        │                   └── overwrite ONLY this field
        └── copy every other field from task first

result = { id: 3, userId: 1, title: "Write notes", completed: true }
```

Wiring it up — `TaskItem` (`src/components/TaskItem.tsx`) gets an `id` prop and an `onToggle` callback prop:

```
<button
  className="task-action-button"
  onClick={() => props.onToggle(props.id)}
>
  {props.statusClass === "completed" ? "Mark Pending" : "Mark Completed"}
</button>
```

`onClick={() => props.onToggle(props.id)}` — `onToggle` needs to know WHICH task, but only this specific `TaskItem` knows its own `id`. Wrapping the call in a small arrow function is how a specific argument gets attached to a click handler — the exact same shape Session 07 already used for `onClick={() => handleSelectPerson(entry.user.id)}`.

8. `.filter()` — Delete

```
function handleDelete(id: number) {
  const updatedTasks = tasks.filter((task) => task.id !== id);
  setTasks(updatedTasks);
}
```

Nothing new here syntactically — this is the exact same `.filter()` already used to build `visibleTasks` in Session 07. "Keep every task whose `id` does NOT match" produces a brand-new array missing exactly one task.

9. Editing a title — the full round trip

Editing needs a question answered first, locally, inside `TaskItem`: **is this row currently showing plain text, or an editable input?** That's local UI state, using plain `useState` — nothing new:

```
const [isEditing, setIsEditing] = useState(false);
const [editTitle, setEditTitle] = useState(props.title);
```

When `isEditing` is `true`, `TaskItem` renders an input and Save/Cancel buttons instead of the normal row. Clicking Save calls a callback prop — the same pattern as Toggle/Delete, just carrying an extra piece of information (the new title text):

```
interface TaskItemProps {
  id: number;
  title: string;
  ownerName: string;
  statusText: string;
```

```
  statusClass: string;
  onToggle: (id: number) => void;
  onDelete: (id: number) => void;
  onSaveEdit: (id: number, newTitle: string) => void;
}
```

```
function handleSaveClick() {
  const trimmedTitle = editTitle.trim();

  if (trimmedTitle !== "") {
    props.onSaveEdit(props.id, trimmedTitle);
  }

  setIsEditing(false);
}
```

And in `App`, `handleSaveEdit` is shaped exactly like `handleToggle` — `.map()` plus object spread — just overwriting a different field:

```
function handleSaveEdit(id: number, newTitle: string) {
  const updatedTasks = tasks.map((task) => {
    if (task.id === id) {
      return { ...task, title: newTitle };
    }

    return task;
  });

  setTasks(updatedTasks);
}
```

Why the trim/empty check lives inside `TaskItem`, not in `App`: the check is about whether the EDIT ITSELF is valid input, which is a question the form-like editing UI should answer before ever asking the parent to change real state — the same reasoning `AddTaskForm` already uses before calling `onAddTask`.

Notice what does NOT change: in `{ ...task, title: newTitle }`, `id`, `userId`, and `completed` all pass through untouched — only `title` is overwritten. This is "Edit TITLE ONLY," exactly as scoped this session. Editing the task's owner is intentionally saved for a later session.

10. Why everything else "just works"

After ANY of Add/Toggle/Delete/Edit, `visibleTasks`, `totalCount`, `completedCount`, `pendingCount`, and `peopleWithCounts` all update correctly — and none of their code changed at all this session. That's because Session 07 already made every one of them **derived**: calculated fresh from `tasks` on every render, never stored separately. The moment `tasks` legitimately changes (via `setTasks`), every derived value recalculates automatically the next time `App` renders. This is the payoff of Session 07's derived-data lesson, now applied to a collection that actually changes over time instead of a fixed sample array.

Glossary

- **Collection state** — state whose value is an array (or another collection), not a single scalar value. `useState<Task[]>` is collection state.
- **Immutable update** — changing state by building a brand-new array/object instead of editing the existing one in place.
- **Array spread (`...array`)** — copies every item of an existing array into a new array literal.
- **Object spread (`...object`)** — copies every field of an existing object into a new object literal; fields written after the spread overwrite the copied ones.
- **Callback prop** — a function passed down as a prop so a child component can ask its parent to do something, without the child needing direct access to the parent's state.
- **CRUD** — Create, Read, Update, Delete — the four basic operations this session implements for tasks (Add = Create, the existing list = Read, Toggle/Edit = Update, Delete = Delete).

Common mistakes

```
// WRONG — mutates the array in place
tasks.push(newTask);
setTasks(tasks);

// RIGHT — builds a brand-new array
setTasks([...tasks, newTask]);
```

```
// WRONG — drops every other field on the task
return { id: task.id, completed: !task.completed };

// RIGHT — copies every field, then overwrites one
```

```
return { ...task, completed: !task.completed };
```

```
// WRONG — forgets the fall-through, breaks every other task
const updatedTasks = tasks.map((task) => {
  if (task.id === id) {
    return { ...task, completed: !task.completed };
  }
  // missing: return task;
});

// RIGHT
const updatedTasks = tasks.map((task) => {
  if (task.id === id) {
    return { ...task, completed: !task.completed };
  }
  return task;
});
```

Review questions

1. Why couldn't Session 07's plain `const tasks` array support Add/Toggle/Delete/Edit?
2. What does the `Task[]` inside `useState<Task[]>` actually mean?
3. What are the four steps in the "ACTION → ... → UI CHANGES" story for this session?
4. What does array spread (`[...tasks, newTask]`) do, in your own words?
5. What does object spread (`{ ...task, completed: !task.completed }`) do, and why is copying every field first important?
6. What is a callback prop, and why does `AddTaskForm` need one instead of touching `tasks` directly?
7. Why does `.map()` 's callback need a `return task;` line for tasks that aren't being changed?
8. Which task fields are allowed to change when you edit a title, and which must stay exactly the same?
9. Why do the stat cards and "X of Y" update correctly after Add/Toggle/Delete without any new code written for them specifically?
10. Why does the empty-title check happen inside `AddTaskForm` / `TaskItem` , rather than inside `App` 's `handleAddTask` / `handleSaveEdit` ?

Answers

1. `const` only stops the variable from being reassigned — it doesn't stop the array from being mutated (`.push()` would still work). The real reason is that a plain variable, mutated or not, never tells React to render again — React only reacts to state changes, and a plain variable isn't state.
2. It tells `useState` what TYPE of value this piece of state holds — an array of `Task` objects, the same interface already used for typed props and typed arrays.
3. ACTION → CREATE A NEW ARRAY → `setTasks(newArray)` → REACT RENDERS AGAIN → derived UI recalculates automatically.
4. It copies every existing item in `tasks` into a new array, then adds `newTask` at the end — the original `tasks` array is never touched.
5. It copies every field already on `task` into a new object, then `completed: !task.completed` overwrites just that one field on the copy. Copying every field first is what preserves `id`, `userId`, and `title` unchanged.
6. A callback prop is a function passed down as a prop so a child can ask its parent to make a change. `AddTaskForm` needs one because it has no access to `App`'s `tasks` state and shouldn't — it only knows how to collect a title and an owner, then ask.
7. Without it, every task that ISN'T being changed would be replaced by `undefined`, since `.map()` uses whatever the callback returns for every single item.
8. `title` is allowed to change. `id`, `userId`, and `completed` must stay exactly the same — object spread guarantees this by copying them first.
9. Because they are DERIVED values (Session 07's pattern) — calculated fresh from `tasks` on every render, not stored separately. Once `tasks` legitimately changes, every derived value automatically recalculates the next render.
10. Because it's a question about whether the SUBMITTED INPUT is valid, which the form/editing UI should decide before ever asking the parent to touch real state — the same reasoning already used for other controlled-input validation.

What you should be able to explain now

- □ Why `tasks` needed to become `useState<Task[]>` instead of staying a `const` array.
- □ The full ACTION → new array → `setTasks` → re-render → derived-UI-updates story, and how it compares to Session 04C's manual mutate-and-render loop.
- □ How array spread builds a new array for Add.
- □ How object spread + `.map()` updates exactly one field on exactly one task, for Toggle and Edit.
- □ How `.filter()` removes exactly one task for Delete.

- □ What a callback prop is and why `AddTaskForm` / `TaskItem` use one instead of touching `tasks` directly.
- □ Why the stat cards, "X of Y", and the people summary never needed new code this session.

Not yet

These ideas are real, and some of them are close relatives of what you just learned — but none of them are taught yet. If you find yourself trying to use one, stop — it belongs to a later session.

- **`useEffect` and fetching real data from a server** — Session 09. `users` and the starting tasks are still plain hardcoded arrays.
- **Deleting with a confirmation prompt** — a later session.
- **Editing a task's owner, or title+owner together** — this session is title-only, on purpose.
- **Keyboard shortcuts for editing (Enter to save, Escape to cancel)** — a later session.
- **Resetting all filters with one button** — a later session.
- **Splitting `App` into many smaller files, or destructuring props** — a later session. Every component in this session still reads props as `props.title`, `props.onDelete`, etc., on purpose.
- **React Fragment** — not used yet anywhere in this course.